2. PERMRAG: THE PROJECT ARCHITECTURE (DEEP DIVE)

Note: You are explaining this from scratch. The panel cannot see your code. Walk them through the problem, the architecture, the decisions, and the metrics.

1. THE NARRATIVE PITCH (90 Seconds)

- **The Problem:** Enterprise LLMs lack native data isolation. A junior analyst shouldn't see the CFO's salary chunk just because they asked a clever question. Standard RAG architectures blindly fetch the most semantically relevant text, creating massive security leaks in multi-tenant or hierarchical environments.
- **The Solution (`PermRAG`):** I built a **Permission-Aware RAG system** from scratch. It enforces a two-dimensional Access Control List (Clearance lattice + Role gates) at the chunk-metadata level, executing the authorization filter *before* the vector space search.
- **The Tech Stack:** LangGraph (Orchestration), PostgreSQL + pgvector (Vector Store), Snowflake Arctic (Embeddings), and a custom Golden Set Evaluation harness.

2. THE ARCHITECTURE (End-to-End Flow)

A. The Offline Ingestion Pipeline

1. **Document Loader:** Reads enterprise PDFs (HR, Finance, Legal).
2. **Text Splitter:** Splits text recursively. *(Decision: 200 chunk size, 30 overlap to preserve straddling facts)*.
3. **ACL Injection:** Before embedding, every single chunk is tagged with JSON metadata (`required_clearance_level`, `allowed_roles`).
4. **Embedding:** Text is converted to 384-dimensional dense vectors using `snowflake-arctic-embed-xs`.
5. **Storage:** Vectors and metadata are committed to `pgvector`.

B. The Online Retrieval & Generation Pipeline

1. **Query Interception:** User submits a prompt. The system intercepts the user's JWT/Session to extract their exact Role and Clearance Level.
2. **Pre-Filtering (The Security Gate):** The system executes a deterministic `WHERE` clause on the vector database metadata. Any chunk the user is not authorized to see is mathematically excluded from the search space.
3. **Vector Search:** An HNSW Approximate Nearest Neighbor search is run *only* on the authorized chunks.
4. **LangGraph Orchestration:** The top chunks are passed into a LangGraph state machine.
5. **The Abstention Check:** If the highest chunk score falls below the confidence threshold, the agent explicitly abstains ("I lack the authorized context to answer this").

6. **Generation:** The context is injected into the LLM prompt, forcing citations.

3. THE ENGINEERING DECISIONS & TRAPS

- **Decision 1: Pre-Filtering vs. Post-Filtering**
  - *The Trap:* Post-filtering searches the vector space first, grabs Top 10, and then drops unauthorized chunks. If 9 chunks were unauthorized, you suffer **"Top-K Starvation"** (passing only 1 chunk to the LLM).
  - *The Fix:* Pre-filtering guarantees that the Top K returned are 100% authorized. (Tradeoff: Requires a database that supports combined vector/metadata indexes).
- **Decision 2: Cosine Similarity vs. Dot Product**
  - *The Trap:* Defaulting to Cosine Similarity because tutorials say so.
  - *The Fix:* Snowflake Arctic embeddings are **L2-Normalized** (length = 1) by default. When vectors are normalized, Cosine and Dot Product yield the exact same ranking order. I chose Dot Product because it requires fewer CPU floating-point operations, saving compute.
- **Decision 3: Why LangGraph instead of a simple Chain?**
  - *The Trap:* Adding an agent "just because."
  - *The Fix:* A standard LangChain LCEL chain is a 1-pass DAG (Directed Acyclic Graph). It cannot loop. I needed a state machine that supports **cycles** so the system could pause, evaluate the confidence threshold, and explicitly route to an abstention branch or a re-query branch.

4. THE METRICS (Real Hardcoded Numbers)

Use these numbers to prove you actually built it. Bluffers never have metrics.

- **Corpus:** 16 enterprise documents resulting in **80 chunks**.
- **Chunking Strategy: 200-token chunks, 30 overlap**. (Chosen via ablation study against Chroma's 800-token default, which proved suboptimal).
- **Embeddings:** 384 dimensions. L2-Normalized.
- **Latency:** Embedding takes **0.86s** (**93 chunks/sec**).
- **Evaluation Harness:** I hand-built a **40-question Golden Dataset**. It includes adversarial `expect_abstain` queries. I evaluated Retrieval Quality (Recall@5) separately from Generation Quality (Faithfulness).